Shopyland

Project Documentation | DreamHacks 2026 | Track 1: Smart Sales, Marketing & E-Commerce

Live demo: https://shopyland-storefront.onrender.com (Render free tier; first load after idle can take ~1 minute to wake)

Prompt: Build a dynamic pricing or recommendation engine that helps local businesses on a remote island sell unique crafts to global online customers while optimizing shipping routes.

Shopyland is a marketplace for remote-island artisan cooperatives. It delivers all three pieces the prompt asks for — a recommendation engine, a dynamic pricing engine, and shipping-route optimization — wired to a live Postgres database that every visitor shares.

1. The three engines

ShopSense recommendation engine. A shopper answers four questions: purpose (gift / personal / community-bulk), usage frequency, household size, and sustainability priority. A deterministic scoring model ranks the catalog and records which signal caused each pick. An LLM (OpenAI gpt-4o-mini) then re-orders the top few and rewrites the reason sentences, but is constrained to cite only the signals the scorer already matched — so every recommendation is traceable with no black box. If the cart already holds items from one artisan hub, the engine prefers products from that same hub so the order ships in a single crate.

Cargo-aware dynamic pricing. There is one shared island-ferry container with a real weight capacity. POST /api/pricing weighs the cart on the server, checks how full this order would leave the returning ferry, and grants a consolidation discount: 10% at 60% full, 20% at 85%, plus a five-unit volume floor. The server returns the final price; the client cannot forge it. On the product page a single quantity slider shows the ShopSense-suggested amount and the seller's own volume tiers, and clamps to the artisan's floor price so a discount never pushes a maker below their minimum.

Shipping-route optimizer. GET /api/logistics/route returns the Artisan → Harbor Hub → Regional Port → Buyer waypoints with real haversine distances and per-leg CO2 figures, drawn on the storefront from the buyer's location.

Multiple cooperatives can list the same craft; buyers compare price, stock, and lead time. Artisans restock with one tap on their store page, which writes straight to Postgres. Place an order and the ferry meter fills in real time, the discount tier flips for every viewer, and a row lands in the database.

2. How the recommendation scoring works

The deterministic model assigns additive weights based on documented purchasing-behaviour directions:

- purpose = community-bulk and product is bulk-available: +3
- purpose = gift and product is a single item: +2

- usage = regular and product is consumable: +3
- usage = one-time and product is durable: +2
- household size ≥ 4 and product is large / bulk: +1
- sustainability priority high and packaging is minimal: scales with the priority value
- sustainability priority high and product is heavy: small penalty

The top six by score form the shortlist. The LLM re-ranks that shortlist and writes each reason, restricted to the matched signals. Without an API key the app falls back to templated copy and returns which path ran. Every call is logged to the recommendation_logs table. A synthetic calibration harness (npm run calibrate) exercises the engine against a generated shopper population; Vitest unit tests cover the prediction math.

3. Architecture & tech stack

- Next.js 15 (App Router) + TypeScript + Tailwind CSS v4
- Supabase Postgres — 9 tables with row-level security; a MemoryDb fallback keeps the app running if the database is unreachable
- OpenAI gpt-4o-mini for recommendation copy (optional; app runs fully without a key)
- jose for HttpOnly JWT session cookies; nodemailer for OTP email (SMTP wired, needs a provider key)
- Deployed on Render with auto-deploy on git push; GET /api/health checks the database and model

4. What is live vs seeded

Live and running: the database, all three engines, order persistence, the shared real-time cargo meter, and the one-tap artisan restock flow.

Seeded sample data: the product catalog and the six cooperatives; product images are generated icons rather than photos; sign-in uses a demo code flow (email delivery switches on with an SMTP key in production).

5. Challenges and what's next

Real OTP email delivery (SMTP is wired, needs a provider key), a fuller shopkeeper listing studio for editing pricing tiers, and moving cargo capacity to per-route ferries rather than one shared container.

Built in one day for DreamHacks 2026. Recommendation engine, dynamic pricing, and route optimization — all live, all explainable, backed by a real database.